BitCoin Indexer

₿ Bitcoin Indexer — Full Architecture & Schema Guide

> **Stack:** Go · PostgreSQL 16 · Apache AGE (optional graph layer) · Bitcoin Core RPC

Architecture Overview

```
Bitcoin Core Node (Full Synced)
        │
        │ ZMQ / RPC (getblock, verbose=2)
        ▼
┌──────────────────────────────────┐
│        Go Indexer Service        │
│  ┌──────────┐  ┌──────────────┐  │
│  │ Block    │  │  UTXO        │  │
│  │ Fetcher  │─►│  Resolver    │  │
│  │ (pool)   │  │  (batch)     │  │
│  └──────────┘  └──────────────┘  │
│       │              │           │
│       ▼              ▼           │
│  ┌──────────────────────────┐    │
│  │   Batch Writer (pgx COPY)│    │
│  │   128-256 blocks/batch   │    │
│  └──────────────────────────┘    │
└──────────────────────────────────┘
        │
        ├── PostgreSQL 16 (Primary DB)
        │     ├── blocks
        │     ├── transactions        ← partitioned by block_height
        │     ├── tx_outputs          ← partitioned by block_height
        │     ├── tx_inputs           ← partitioned by block_height
        │     ├── utxo_set            ← HOT table, always current
        │     ├── address_transactions← denormalized, partitioned
        │     └── address_balances    ← cached, updated in batch
```

```
          │
          └── Apache AGE (optional)
                └── address → tx → address graph for tracing
```

Key design decision: Two-phase ingestion — outputs first, then mark-spent in bulk. This eliminates the Neo4j sequential UTXO lookup bottleneck entirely.

Why PostgreSQL over Neo4j

Problem	Neo4j	PostgreSQL
UTXO lookup on insert	Sequential, node-lock	Batch UPDATE with JOIN
Balance update	Per-node transaction	Bulk CASE WHEN update
10k address tx query	Cypher graph walk (slow)	Pre-indexed `address_transactions` row scan
BRIN index support	❌ None	✅ Perfect for block_height
Free tier scale	200k nodes free	Unlimited on Supabase/Neon free
Go ecosystem	Neo4j driver	pgx (fastest, native COPY protocol)
Concurrent writes	Writer bottleneck	Parallel partition inserts

The core insight: Neo4j is slow here because each insert triggers a graph traversal to find and update the UTXO node. PostgreSQL lets you batch all 500 UTXOs from a block, insert them, then do a single bulk `UPDATE tx_outputs SET is_spent = true WHERE (txid, vout) IN (...)` — that's a hash join, not a loop.

Core Schema Design

Design Principles

- **Row-wise = column layout matters.** Put the most-queried columns first in each table. PostgreSQL reads full rows on heap scan, so keep hot columns together.
- **Denormalize `address_transactions`** — this is your Arkham-equivalent "address history" table. Never JOIN to get address tx history.

- `utxo_set` **is your hot materialized view** — updated every block. Never compute current UTXO from `tx_outputs` in production queries.
- **BRIN indexes on** `block_height` — block height is monotonically increasing. BRIN covers 128 blocks per page in 1/1000th the space of BTREE.

Table: `blocks`

```
CREATE TABLE blocks (
    height          INT             PRIMARY KEY,
    hash            BYTEA           NOT NULL UNIQUE,        -
- 32 bytes
    prev_hash       BYTEA           NOT NULL,
    merkle_root     BYTEA           NOT NULL,
    block_time      TIMESTAMPTZ     NOT NULL,
    bits            BIGINT          NOT NULL,
    nonce           BIGINT          NOT NULL,
    version         INT             NOT NULL,
    tx_count        INT             NOT NULL,
    size_bytes      INT             NOT NULL,
    weight          INT             NOT NULL,
    total_fees_sats BIGINT          NOT NULL DEFAULT 0,
    indexed_at      TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);
```

Table: `transactions`

```
CREATE TABLE transactions (
    txid            BYTEA           NOT NULL,               -
- 32 bytes, primary key
    block_height    INT             NOT NULL,
    block_hash      BYTEA           NOT NULL,
    tx_index        INT             NOT NULL,               -
- position in block
    version         INT             NOT NULL,
    locktime        BIGINT          NOT NULL,
```

```
    is_coinbase     BOOL            NOT NULL DEFAULT FALSE,
    input_count     SMALLINT        NOT NULL,
    output_count    SMALLINT        NOT NULL,
    fee_sats        BIGINT,                                 -
- NULL until inputs resolved
    size_bytes      INT,
    vsize           INT,                                    -
- virtual size (segwit)
    weight          INT,
    has_segwit      BOOL            NOT NULL DEFAULT FALSE,
    PRIMARY KEY (txid)
) PARTITION BY RANGE (block_height);
```

Table: `tx_outputs` (UTXO source of truth)

```
CREATE TABLE tx_outputs (
    txid            BYTEA           NOT NULL,
    vout_idx        INT             NOT NULL,
    block_height    INT             NOT NULL,               -
- for partitioning
    value_sats      BIGINT          NOT NULL,
    script_pubkey   BYTEA           NOT NULL,
    script_type     SMALLINT        NOT NULL,               -
- enum below
    address         TEXT,                                   -
- NULL for unspendable
    -- Spend tracking (updated when spending tx is indexed)
    is_spent        BOOL            NOT NULL DEFAULT FALSE,
    spending_txid   BYTEA,
    spending_vin    INT,
    spent_height    INT,
    PRIMARY KEY (txid, vout_idx)
) PARTITION BY RANGE (block_height);

-- script_type enum values:
```

```
-- 0 = P2PKH, 1 = P2SH, 2 = P2WPKH, 3 = P2WSH
-- 4 = P2TR (taproot), 5 = OP_RETURN, 6 = Multisig, 7 = Unkno
wn
```

Table: `tx_inputs`

```
CREATE TABLE tx_inputs (
    txid            BYTEA           NOT NULL,
    vin_idx         INT             NOT NULL,
    block_height    INT             NOT NULL,               -
- for partitioning
    prev_txid       BYTEA,                                  -
- NULL = coinbase
    prev_vout       INT,                                    -
- NULL = coinbase
    script_sig      BYTEA,
    witness_data    BYTEA[],                                -
- array of witness items
    sequence_no     BIGINT          NOT NULL,
    PRIMARY KEY (txid, vin_idx)
) PARTITION BY RANGE (block_height);
```

Table: `address_transactions` ⭐ (The fast-path table)

```
-- This is the single most important table for your use case.
-- Every time an address appears in a tx (as sender OR receiv
er),
-- write a row here. Queries for "last 10k tx of address X" h
it
-- only this table — zero joins.
CREATE TABLE address_transactions (
    address         TEXT            NOT NULL,
    block_height    INT             NOT NULL,
    tx_index        INT             NOT NULL,               -
- for stable sort within block
```

```
    txid            BYTEA           NOT NULL,
    role            SMALLINT        NOT NULL,               -
- 0=receiver, 1=sender, 2=both
    net_value_sats  BIGINT          NOT NULL,               -
- positive=received, negative=sent
    PRIMARY KEY (address, block_height DESC, tx_index, txid)
) PARTITION BY RANGE (block_height);
-- DESC on block_height so latest rows come first in the PK s
can
-- "Last 10k tx" = forward scan on index, no sort needed
```

Table: `utxo_set` ⭐ (Hot materialized UTXO state)

```
-- This is your live UTXO set. Keep it small and fast.
-- Never JOIN tx_outputs for "current balance" — use this.
CREATE TABLE utxo_set (
    txid            BYTEA           NOT NULL,
    vout_idx        INT             NOT NULL,
    address         TEXT            NOT NULL,
    value_sats      BIGINT          NOT NULL,
    block_height    INT             NOT NULL,
    script_type     SMALLINT        NOT NULL,
    PRIMARY KEY (txid, vout_idx)
);
```

Table: `address_balances` (Cached balances)

```
CREATE TABLE address_balances (
    address             TEXT        PRIMARY KEY,
    balance_sats        BIGINT      NOT NULL DEFAULT 0,
    total_received_sats BIGINT      NOT NULL DEFAULT 0,
    total_sent_sats     BIGINT      NOT NULL DEFAULT 0,
    utxo_count          INT         NOT NULL DEFAULT 0,
    tx_count            INT         NOT NULL DEFAULT 0,
    first_seen_height   INT,
```

```
    last_seen_height    INT,
    updated_at          INT         NOT NULL DEFAULT 0
-- block_height of last update
);
```

Index Strategy

BRIN Indexes — Why They're Perfect Here

BRIN (Block Range INdex) stores only the MIN and MAX value of a column per 128-page heap block. For `block_height`, which is written in order (block 1 then 2 then 3...), BRIN is effectively lossless and 1000x smaller than BTREE.

Rule: Use BRIN for `block_height` on all large tables. Use BTREE for `txid`, `address`, and any column you query by exact value.

```
-- ============================================================
==
-- BRIN indexes (block_height — monotonically increasing)
-- ============================================================
==
CREATE INDEX CONCURRENTLY idx_tx_height_brin
    ON transactions USING BRIN (block_height) WITH (pages_per
_range = 32);

CREATE INDEX CONCURRENTLY idx_txout_height_brin
    ON tx_outputs USING BRIN (block_height) WITH (pages_per_r
ange = 32);

CREATE INDEX CONCURRENTLY idx_txin_height_brin
    ON tx_inputs USING BRIN (block_height) WITH (pages_per_ra
nge = 32);

CREATE INDEX CONCURRENTLY idx_addrtx_height_brin
    ON address_transactions USING BRIN (block_height) WITH (p
ages_per_range = 32);
```

```sql
-- ============================================================
==
-- BTREE indexes (exact lookups)
-- ============================================================
==

-- Fast txid lookup (most common query)
CREATE INDEX CONCURRENTLY idx_tx_txid
    ON transactions (txid);

-- Output lookup by txid (for UTXO resolution)
CREATE INDEX CONCURRENTLY idx_txout_txid
    ON tx_outputs (txid);

-- Unspent outputs by address (for balance / UTXO queries)
CREATE INDEX CONCURRENTLY idx_txout_addr_unspent
    ON tx_outputs (address, value_sats DESC)
    WHERE is_spent = FALSE AND address IS NOT NULL;

-- UTXO set by address (for balance lookup)
CREATE INDEX CONCURRENTLY idx_utxo_address
    ON utxo_set (address, value_sats DESC);

-- Spending tx lookup (for tracing)
CREATE INDEX CONCURRENTLY idx_txout_spending
    ON tx_outputs (spending_txid)
    WHERE spending_txid IS NOT NULL;

-- Address transaction history (THE main query path)
-- PK already covers (address, block_height, tx_index, txid)
— this is your index.
-- Add a separate index for address-only prefix scans:
CREATE INDEX CONCURRENTLY idx_addrtx_addr
    ON address_transactions (address, block_height DESC);
```

```
-- Input prev_txid lookup (for tracing input→output)
CREATE INDEX CONCURRENTLY idx_txin_prev
    ON tx_inputs (prev_txid, prev_vout)
    WHERE prev_txid IS NOT NULL;

-- Block hash lookup
CREATE INDEX CONCURRENTLY idx_blocks_hash
    ON blocks (hash);

-- ============================================================
==
-- GIN index (for future JSONB/array witness data, optional)
-- ============================================================
==
-- CREATE INDEX CONCURRENTLY idx_txin_witness
--     ON tx_inputs USING GIN (witness_data);
```

Can You Use BRIN on Everything?

No. BRIN only helps when the data is **physically ordered** on disk the same way it's logically ordered. Here's the decision table:

Column	Index Type	Why
`block_height`	BRIN	Appended in order, perfect correlation
`txid`	BTREE	Random 32-byte hash, no correlation
`address`	BTREE	Arbitrary strings, no correlation
`is_spent`	Partial BTREE	Low cardinality, partial is better
`value_sats`	BTREE	Only if you query ranges
`block_time`	BRIN	Monotonically increasing with height

Partitioning Strategy

Partition all large tables by `block_height` in ranges of 100,000 blocks (~2 years of Bitcoin). This keeps each partition ~50-80GB max and allows partition pruning on range queries.

```sql
-- Create partitions for transactions (and same for tx_output
s, tx_inputs, address_transactions)
CREATE TABLE transactions_0_100k
    PARTITION OF transactions FOR VALUES FROM (0) TO (10000
0);

CREATE TABLE transactions_100k_200k
    PARTITION OF transactions FOR VALUES FROM (100000) TO (20
0000);

CREATE TABLE transactions_200k_300k
    PARTITION OF transactions FOR VALUES FROM (200000) TO (30
0000);

CREATE TABLE transactions_300k_400k
    PARTITION OF transactions FOR VALUES FROM (300000) TO (40
0000);

CREATE TABLE transactions_400k_500k
    PARTITION OF transactions FOR VALUES FROM (400000) TO (50
0000);

CREATE TABLE transactions_500k_600k
    PARTITION OF transactions FOR VALUES FROM (500000) TO (60
0000);

CREATE TABLE transactions_600k_700k
    PARTITION OF transactions FOR VALUES FROM (600000) TO (70
0000);

CREATE TABLE transactions_700k_800k
    PARTITION OF transactions FOR VALUES FROM (700000) TO (80
0000);

-- Current partition (~850k as of 2026)
```

```sql
CREATE TABLE transactions_800k_900k
    PARTITION OF transactions FOR VALUES FROM (800000) TO (90
0000);

CREATE TABLE transactions_900k_1m
    PARTITION OF transactions FOR VALUES FROM (900000) TO (10
00000);
```

Repeat the same partition ranges for `tx_outputs`, `tx_inputs`, and `address_transactions`.

Partition Pruning in Action

```sql
-- This query hits ONLY the 800k-900k partition:
SELECT txid FROM transactions
WHERE block_height BETWEEN 850000 AND 860000;

-- PostgreSQL plan will show: Seq Scan on transactions_800k_9
00k
-- All other partitions are pruned automatically.
```

Ingestion Pipeline (Go)

The Bottleneck You're Hitting in Neo4j

Neo4j's problem: for each input, you lookup the UTXO → update it → mark spent. That's a graph traversal per input, serialized. At 5000+ tx/block, this is catastrophically slow.

PostgreSQL Solution: Two-Phase Batch Ingestion

```
Phase 1: INSERT all outputs (bulk COPY)
Phase 2: UPDATE spent status (single bulk join UPDATE)

No row-by-row processing. No sequential UTXO lookup.
```

Go Architecture

```go
package indexer

import (
    "context"
    "github.com/jackc/pgx/v5/pgxpool"
    "github.com/jackc/pgx/v5"
)

type Indexer struct {
    db       *pgxpool.Pool
    rpc      *BitcoinRPC
    batchSz  int           // 128 or 256 blocks per batch
    workers  int           // parallel goroutines
}

// BlockBatch holds all data from N blocks before DB write
type BlockBatch struct {
    Blocks       []Block
    Transactions []Transaction
    Outputs      []TxOutput
    Inputs       []TxInput
    AddrTxRows   []AddressTransaction
}
```

Key: Use pgx COPY Protocol (10x faster than INSERT)

```go
func (idx *Indexer) writeBatch(ctx context.Context, batch *Bl
ockBatch) error {
    conn, err := idx.db.Acquire(ctx)
    defer conn.Release()

    // --- Phase 1: COPY all outputs ---
    _, err = conn.CopyFrom(
        ctx,
```

```
        pgx.Identifier{"tx_outputs"},
        []string{"txid", "vout_idx", "block_height", "value_s
ats",
                 "script_pubkey", "script_type", "address"},
        pgx.CopyFromSlice(len(batch.Outputs), func(i int) ([]
any, error) {
            o := batch.Outputs[i]
            return []any{o.Txid, o.VoutIdx, o.BlockHeight, o.
ValueSats,
                         o.ScriptPubkey, o.ScriptType, o.Addr
ess}, nil
        }),
    )

    // --- Phase 2: Bulk mark-spent (single UPDATE, not N UPD
ATEs) ---
    // Build temp table with (prev_txid, prev_vout, spending_
txid, vin_idx, spent_height)
    _, err = conn.Exec(ctx, `CREATE TEMP TABLE _spent_inputs
(
        prev_txid     BYTEA, prev_vout INT,
        spending_txid BYTEA, spending_vin INT, spent_height I
NT
    ) ON COMMIT DROP`)

    conn.CopyFrom(ctx, pgx.Identifier{"_spent_inputs"}, ...)

    // Single bulk UPDATE — PostgreSQL hash join, not a loop
    _, err = conn.Exec(ctx, `
        UPDATE tx_outputs o
        SET    is_spent = TRUE,
               spending_txid = s.spending_txid,
               spending_vin  = s.spending_vin,
               spent_height  = s.spent_height
        FROM   _spent_inputs s
        WHERE  o.txid = s.prev_txid AND o.vout_idx = s.prev_v
```

```
out
    `)

    // --- Phase 3: Update utxo_set ---
    // Delete spent UTXOs
    _, err = conn.Exec(ctx, `
        DELETE FROM utxo_set u
        USING _spent_inputs s
        WHERE u.txid = s.prev_txid AND u.vout_idx = s.prev_vo
ut
    `)

    // Insert new UTXOs
    conn.CopyFrom(ctx, pgx.Identifier{"utxo_set"}, ...)

    // --- Phase 4: Upsert address_balances ---
    _, err = conn.Exec(ctx, `
        INSERT INTO address_balances (address, balance_sats,
total_received_sats,
                                      total_sent_sats, utxo_c
ount, tx_count,
                                      first_seen_height, last
_seen_height)
        SELECT
            address,
            SUM(CASE WHEN is_spent THEN 0 ELSE value_sats EN
D),
            SUM(value_sats),
            SUM(CASE WHEN is_spent THEN value_sats ELSE 0 EN
D),
            COUNT(*) FILTER (WHERE NOT is_spent),
            COUNT(DISTINCT txid),
            MIN(block_height),
            MAX(block_height)
        FROM tx_outputs
        WHERE address IN (SELECT DISTINCT address FROM _batch
```

```
_addresses)
        GROUP BY address
        ON CONFLICT (address) DO UPDATE SET
            balance_sats        = EXCLUDED.balance_sats,
            total_received_sats = EXCLUDED.total_received_sat
s,
            total_sent_sats     = EXCLUDED.total_sent_sats,
            utxo_count          = EXCLUDED.utxo_count,
            tx_count            = EXCLUDED.tx_count,
            last_seen_height    = EXCLUDED.last_seen_height
    `)

    return nil
}
```

Parallel Worker Pool

```go
func (idx *Indexer) Run(ctx context.Context, startHeight int)
error {
    blockCh := make(chan int, idx.workers*4)
    batchCh := make(chan *BlockBatch, 4)

    // Producer: emit block heights
    go func() {
        for h := startHeight; ; h++ {
            blockCh <- h
        }
    }()

    // N workers: fetch + parse blocks in parallel
    var wg sync.WaitGroup
    for i := 0; i < idx.workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
```

```go
            batch := &BlockBatch{}
            for h := range blockCh {
                block, err := idx.rpc.GetBlock(ctx, h)
                idx.parseBlock(block, batch)
                if len(batch.Blocks) >= idx.batchSz {
                    batchCh <- batch
                    batch = &BlockBatch{}
                }
            }
        }()
    }

    // Single writer: ordered DB writes
    for batch := range batchCh {
        idx.writeBatch(ctx, batch)
    }
    return nil
}
```

PostgreSQL Configuration for Bulk Ingestion

```
# postgresql.conf — tuning for initial sync

# Memory (adjust to your RAM; these for 16GB)
shared_buffers            = 4GB
effective_cache_size      = 12GB
work_mem                  = 256MB        # per sort/hash operation
maintenance_work_mem      = 2GB          # for index builds

# WAL / Checkpoints (CRITICAL for bulk insert speed)
wal_level                 = minimal      # during initial sync only
max_wal_size              = 8GB
checkpoint_completion_target = 0.9
checkpoint_timeout        = 30min

# Parallelism
max_parallel_workers_per_gather = 4
max_parallel_workers      = 8
max_worker_processes      = 10

# Disable fsync ONLY during initial historical sync, re-enable after!
```

```
# fsync = off               # DANGER: only for bulk load phase
# synchronous_commit = off  # Safer alternative; re-enable after sync

# Connection pooling (use PgBouncer or pgx pool)
max_connections           = 100
```

UTXO Management

The UTXO Update Problem (Why Neo4j Fails)

Neo4j finds inputs sequentially → for each input, traverses to find the UTXO node → acquires write lock → updates `is_spent`. This is O(n) graph traversals per block, serialized. At Bitcoin's scale (1B+ UTXOs), this is the death spiral.

PostgreSQL Solution: Separate UTXO Hot Table

```
tx_outputs    = immutable historical record (never update is_spent here during sync)
utxo_set      = ephemeral, always-current UTXO state (bulk replace each batch)
```

During initial sync, you can delay marking `is_spent` on `tx_outputs` entirely and use only the `utxo_set` table. After full sync, run a one-time backfill:

```
-- One-time backfill: mark all spent outputs
UPDATE tx_outputs o
SET    is_spent      = TRUE,
       spending_txid = i.txid,
       spending_vin  = i.vin_idx,
       spent_height  = t.block_height
FROM   tx_inputs i
JOIN   transactions t ON t.txid = i.txid
WHERE  o.txid = i.prev_txid AND o.vout_idx = i.prev_vout
  AND  i.prev_txid IS NOT NULL;
-- Run with parallel workers, will take 2-4 hours on full cha
in
```

UTXO Set Size Reference

Year	Block Height	UTXO Set Size	Row Count
2015	~400k	~3GB	~45M rows
2020	~650k	~5GB	~68M rows
2026	~870k	~11GB	~180M rows

Query Patterns

1. Last 10,000 transactions for an address

```
-- Hits ONLY address_transactions, zero joins, index scan on
(address, block_height DESC)
SELECT txid, block_height, tx_index, role, net_value_sats
FROM   address_transactions
WHERE  address = '1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf'
ORDER  BY block_height DESC, tx_index DESC
LIMIT  10000;
-- Target: < 20ms with warm cache
```

2. Current balance of an address

```
SELECT SUM(value_sats) AS balance_sats, COUNT(*) AS utxo_coun
t
FROM   utxo_set
WHERE  address = '1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf';
-- Target: < 5ms (index scan on utxo_set)

-- Or even faster with cached table:
SELECT balance_sats, utxo_count, tx_count
FROM   address_balances
WHERE  address = '1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf';
-- Target: < 1ms (single row lookup)
```

3. Trace a transaction (inputs → outputs)

```
WITH tx_ins AS (
    SELECT i.prev_txid, i.prev_vout, o.address AS from_addr,
o.value_sats AS in_value
    FROM   tx_inputs i
    JOIN   tx_outputs o ON o.txid = i.prev_txid AND o.vout_id
x = i.prev_vout
    WHERE  i.txid = '\xabc123...'::bytea
),
tx_outs AS (
    SELECT vout_idx, address AS to_addr, value_sats AS out_va
lue, is_spent
    FROM   tx_outputs
    WHERE  txid = '\xabc123...'::bytea
)
SELECT * FROM tx_ins, tx_outs;
```

4. Find connected wallets (1-hop tracing)

```
-- All addresses that ever co-spent inputs with target addres
s
-- (Heuristic: common input ownership)
SELECT DISTINCT o2.address
FROM   tx_inputs i1
JOIN   tx_inputs i2  ON i2.txid = i1.txid AND i2.vin_idx != i
1.vin_idx
JOIN   tx_outputs o1 ON o1.txid = i1.prev_txid AND o1.vout_id
x = i1.prev_vout
JOIN   tx_outputs o2 ON o2.txid = i2.prev_txid AND o2.vout_id
x = i2.prev_vout
WHERE  o1.address = '1Target...'
  AND  o2.address IS NOT NULL
  AND  o2.address != '1Target...'
LIMIT  1000;
```

5. Transactions in a block range

```
-- Partition pruning kicks in automatically
SELECT txid, fee_sats, vsize, has_segwit
FROM   transactions
WHERE  block_height BETWEEN 840000 AND 840100
ORDER  BY block_height, tx_index;
```

6. Largest transactions by value in last N blocks

```
SELECT t.txid, t.block_height, SUM(o.value_sats) AS total_out
put_sats
FROM   transactions t
JOIN   tx_outputs o ON o.txid = t.txid
WHERE  t.block_height > (SELECT MAX(height) - 1000 FROM block
s)
GROUP  BY t.txid, t.block_height
ORDER  BY total_output_sats DESC
LIMIT  100;
```

Apache AGE Graph Layer

Use Apache AGE **only** for multi-hop wallet tracing (2+ hops) where the SQL recursive CTE becomes unwieldy. Do NOT use AGE for basic queries — PostgreSQL handles those faster.

When to Use AGE

- 3+ hop address clustering
- Finding all addresses connected to a blacklisted wallet
- Identifying exchange deposit patterns (many→one)
- Peel chain analysis

Schema in AGE

```
-- Enable AGE extension
CREATE EXTENSION IF NOT EXISTS age;
LOAD 'age';
SET search_path = ag_catalog, "$user", public;

-- Create Bitcoin graph
SELECT create_graph('bitcoin');

-- Create vertex labels
SELECT create_vlabel('bitcoin', 'Address');
SELECT create_vlabel('bitcoin', 'Transaction');

-- Create edge labels
SELECT create_elabel('bitcoin', 'SENT');      -- Address → Tr
ansaction
SELECT create_elabel('bitcoin', 'RECEIVED');  -- Transaction
→ Address
SELECT create_elabel('bitcoin', 'CLUSTER');   -- Address → Ad
dress (same wallet heuristic)
```

Populating AGE from PostgreSQL (Go)

```
// Sync AGE graph in parallel with PostgreSQL inserts
// Only populate AGE if address is new (deduplicate at app la
yer)
func (idx *Indexer) writeToAGE(ctx context.Context, batch *Bl
ockBatch) error {
    for _, tx := range batch.Transactions {
        // Create Transaction vertex
        idx.db.Exec(ctx, `
            SELECT * FROM cypher('bitcoin', $$
                CREATE (t:Transaction {txid: $txid, height:
$height, fee: $fee})
            $$) AS (v agtype)
```

```
        `, tx.Txid, tx.BlockHeight, tx.FeeSats)

        // Create Address vertices + SENT edges for inputs
        for _, inp := range tx.Inputs {
            idx.db.Exec(ctx, `
                SELECT * FROM cypher('bitcoin', $$
                    MATCH (a:Address {addr: $addr})
                    MATCH (t:Transaction {txid: $txid})
                    CREATE (a)-[:SENT {value: $val}]->(t)
                $$) AS (v agtype)
            `, inp.FromAddress, tx.Txid, inp.Value)
        }
    }
    return nil
}
```

AGE Tracing Query: 3-Hop Wallet Cluster

```
-- Find all addresses within 3 hops of target (connected through shared transactions)
SELECT * FROM cypher('bitcoin', $$
    MATCH path = (start:Address {addr: '1Target...'})-
                 [:SENT|RECEIVED*1..6]-
                 (connected:Address)
    WHERE start <> connected
    RETURN DISTINCT connected.addr, length(path) as hops
    ORDER BY hops ASC
    LIMIT 500
$$) AS (addr agtype, hops agtype);
```

Free Deployment Options

Option 1: Supabase (Recommended — PostgreSQL 15, 8GB storage free)

```
# 1. Create project at supabase.com
# 2. Run schema SQL in SQL editor
```

```
# 3. Set env vars in your Go app
DATABASE_URL="postgresql://postgres:[password]@db.[project].s
upabase.co:5432/postgres"

# Supabase free tier: 8GB storage, 2 CPU, 1GB RAM
# Paid tier ($25/mo): 100GB, 8GB RAM — enough for full chain
```

Supabase limitation: 8GB free is only enough for ~blocks 0–400k. For full chain you need paid tier or self-host.

Option 2: Neon.tech (Serverless PostgreSQL, 10GB free)

```
DATABASE_URL="postgresql://user:pass@ep-xxx.us-east-1.aws.neo
n.tech/btcindexer"
# Autoscale to 0, 10GB free storage
# Better for dev/testing than production full-chain indexing
```

Option 3: Self-Host on Railway / Render / Fly.io

```
# Dockerfile for Go indexer
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY . .
RUN go build -o indexer ./cmd/indexer

FROM alpine:latest
COPY --from=builder /app/indexer /usr/local/bin/
CMD ["indexer", "--rpc=http://btcd:8332", "--db=$DATABASE_UR
L"]
```

```
# fly.toml — Fly.io free tier (3 shared VMs free)
app = "btc-indexer"
[build]
  dockerfile = "Dockerfile"
```

```
[env]
  RPC_URL = "http://your-bitcoin-node:8332"
```

Option 4: Run Everything Local + Apache AGE

```yaml
# docker-compose.yml
version: '3.9'
services:
  postgres:
    image: apache/age:PG16_latest         # PostgreSQL 16 + A
GE extension bundled
    environment:
      POSTGRES_DB: btcindexer
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    deploy:
      resources:
        limits:
          memory: 8G

  indexer:
    build: .
    environment:
      DATABASE_URL: "postgres://postgres:secret@postgres:543
2/btcindexer"
      RPC_URL: "http://host.docker.internal:8332"  # your loc
al Bitcoin node
    depends_on:
      - postgres

volumes:
  pgdata:
```

```
    driver: local
    driver_opts:
      type: none
      o: bind
      device: /mnt/ssd/pgdata     # put on SSD!
```

Performance Targets

Operation	Target	How
Initial sync speed	2,000-5,000 blocks/min	8-16 parallel workers + COPY protocol
Last 10k tx of address	< 20ms	`address_transactions` index scan
Current balance	< 2ms	`address_balances` PK lookup
Full tx detail (1 tx)	< 5ms	txid BTREE lookup
1-hop wallet cluster	< 50ms	SQL with index joins
3-hop cluster (AGE)	< 500ms	Cypher graph traversal
Write throughput	50k-100k rows/sec	pgx COPY, batched

Estimated Storage (Full Chain ~870k blocks, 2026)

Table	Rows	Estimated Size
blocks	870k	~400MB
transactions	1.1B	~150GB
tx_outputs	2.5B	~400GB
tx_inputs	2.5B	~350GB
address_transactions	5B	~500GB
utxo_set	180M	~15GB
address_balances	50M	~5GB
Total		**~1.4TB**

For a lighter setup (last 3 years only, ~400k-870k blocks), you'd need ~600GB.

Full SQL Script

```
-- ============================================================
======
-- Bitcoin Indexer Schema — Full Copy-Paste Script
-- PostgreSQL 16 + Apache AGE (optional)
-- ============================================================
======

-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS pg_trgm;       -- for address
text search
CREATE EXTENSION IF NOT EXISTS btree_gin;     -- for GIN on b
asic types
-- CREATE EXTENSION IF NOT EXISTS age;        -- uncomment if
using AGE

-- ============================================================
======
-- 1. BLOCKS
-- ============================================================
======
CREATE TABLE IF NOT EXISTS blocks (
    height              INT             PRIMARY KEY,
    hash                BYTEA           NOT NULL UNIQUE,
    prev_hash           BYTEA           NOT NULL,
    merkle_root         BYTEA           NOT NULL,
    block_time          TIMESTAMPTZ     NOT NULL,
    bits                BIGINT          NOT NULL,
    nonce               BIGINT          NOT NULL,
    version             INT             NOT NULL,
    tx_count            INT             NOT NULL,
    size_bytes          INT             NOT NULL,
    weight              INT             NOT NULL,
    total_fees_sats     BIGINT          NOT NULL DEFAULT 0,
    indexed_at          TIMESTAMPTZ     NOT NULL DEFAULT NOW
()
```

```
);

CREATE INDEX IF NOT EXISTS idx_blocks_hash ON blocks (hash);
CREATE INDEX IF NOT EXISTS idx_blocks_time ON blocks USING BR
IN (block_time);

-- ============================================================
======
-- 2. TRANSACTIONS (partitioned)
-- ============================================================
======
CREATE TABLE IF NOT EXISTS transactions (
    txid            BYTEA           NOT NULL,
    block_height    INT             NOT NULL,
    block_hash      BYTEA           NOT NULL,
    tx_index        INT             NOT NULL,
    version         INT             NOT NULL,
    locktime        BIGINT          NOT NULL,
    is_coinbase     BOOL            NOT NULL DEFAULT FALSE,
    input_count     SMALLINT        NOT NULL DEFAULT 0,
    output_count    SMALLINT        NOT NULL DEFAULT 0,
    fee_sats        BIGINT,
    size_bytes      INT,
    vsize           INT,
    weight          INT,
    has_segwit      BOOL            NOT NULL DEFAULT FALSE,
    PRIMARY KEY (txid)
) PARTITION BY RANGE (block_height);

CREATE TABLE IF NOT EXISTS transactions_0_100k   PARTITION OF
transactions FOR VALUES FROM (0)       TO (100000);
CREATE TABLE IF NOT EXISTS transactions_100k_200k PARTITION O
F transactions FOR VALUES FROM (100000)  TO (200000);
CREATE TABLE IF NOT EXISTS transactions_200k_300k PARTITION O
F transactions FOR VALUES FROM (200000)  TO (300000);
CREATE TABLE IF NOT EXISTS transactions_300k_400k PARTITION O
```

```
F transactions FOR VALUES FROM (300000)  TO (400000);
CREATE TABLE IF NOT EXISTS transactions_400k_500k PARTITION O
F transactions FOR VALUES FROM (400000)  TO (500000);
CREATE TABLE IF NOT EXISTS transactions_500k_600k PARTITION O
F transactions FOR VALUES FROM (500000)  TO (600000);
CREATE TABLE IF NOT EXISTS transactions_600k_700k PARTITION O
F transactions FOR VALUES FROM (600000)  TO (700000);
CREATE TABLE IF NOT EXISTS transactions_700k_800k PARTITION O
F transactions FOR VALUES FROM (700000)  TO (800000);
CREATE TABLE IF NOT EXISTS transactions_800k_900k PARTITION O
F transactions FOR VALUES FROM (800000)  TO (900000);
CREATE TABLE IF NOT EXISTS transactions_900k_1m   PARTITION O
F transactions FOR VALUES FROM (900000)  TO (1000000);

CREATE INDEX IF NOT EXISTS idx_tx_txid   ON transactions (txi
d);
CREATE INDEX IF NOT EXISTS idx_tx_height_brin ON transactions
USING BRIN (block_height) WITH (pages_per_range=32);
CREATE INDEX IF NOT EXISTS idx_tx_coinbase ON transactions (b
lock_height) WHERE is_coinbase = TRUE;

-- ============================================================
======
-- 3. TX_OUTPUTS (partitioned)
-- ============================================================
======
CREATE TABLE IF NOT EXISTS tx_outputs (
    txid            BYTEA           NOT NULL,
    vout_idx        INT             NOT NULL,
    block_height    INT             NOT NULL,
    value_sats      BIGINT          NOT NULL,
    script_pubkey   BYTEA           NOT NULL,
    script_type     SMALLINT        NOT NULL DEFAULT 0,
    address         TEXT,
    is_spent        BOOL            NOT NULL DEFAULT FALSE,
    spending_txid   BYTEA,
```

```sql
    spending_vin    INT,
    spent_height    INT,
    PRIMARY KEY (txid, vout_idx)
) PARTITION BY RANGE (block_height);

CREATE TABLE IF NOT EXISTS tx_outputs_0_100k    PARTITION OF
tx_outputs FOR VALUES FROM (0)       TO (100000);
CREATE TABLE IF NOT EXISTS tx_outputs_100k_200k  PARTITION OF
tx_outputs FOR VALUES FROM (100000)  TO (200000);
CREATE TABLE IF NOT EXISTS tx_outputs_200k_300k  PARTITION OF
tx_outputs FOR VALUES FROM (200000)  TO (300000);
CREATE TABLE IF NOT EXISTS tx_outputs_300k_400k  PARTITION OF
tx_outputs FOR VALUES FROM (300000)  TO (400000);
CREATE TABLE IF NOT EXISTS tx_outputs_400k_500k  PARTITION OF
tx_outputs FOR VALUES FROM (400000)  TO (500000);
CREATE TABLE IF NOT EXISTS tx_outputs_500k_600k  PARTITION OF
tx_outputs FOR VALUES FROM (500000)  TO (600000);
CREATE TABLE IF NOT EXISTS tx_outputs_600k_700k  PARTITION OF
tx_outputs FOR VALUES FROM (600000)  TO (700000);
CREATE TABLE IF NOT EXISTS tx_outputs_700k_800k  PARTITION OF
tx_outputs FOR VALUES FROM (700000)  TO (800000);
CREATE TABLE IF NOT EXISTS tx_outputs_800k_900k  PARTITION OF
tx_outputs FOR VALUES FROM (800000)  TO (900000);
CREATE TABLE IF NOT EXISTS tx_outputs_900k_1m    PARTITION OF
tx_outputs FOR VALUES FROM (900000)  TO (1000000);

CREATE INDEX IF NOT EXISTS idx_txout_txid   ON tx_outputs (tx
id);
CREATE INDEX IF NOT EXISTS idx_txout_height_brin ON tx_output
s USING BRIN (block_height) WITH (pages_per_range=32);
CREATE INDEX IF NOT EXISTS idx_txout_addr_unspent ON tx_outpu
ts (address, value_sats DESC) WHERE is_spent = FALSE AND addr
ess IS NOT NULL;
CREATE INDEX IF NOT EXISTS idx_txout_spending ON tx_outputs
(spending_txid) WHERE spending_txid IS NOT NULL;
```

```
-- ============================================================
======
-- 4. TX_INPUTS (partitioned)
-- ============================================================
======
CREATE TABLE IF NOT EXISTS tx_inputs (
    txid            BYTEA           NOT NULL,
    vin_idx         INT             NOT NULL,
    block_height    INT             NOT NULL,
    prev_txid       BYTEA,
    prev_vout       INT,
    script_sig      BYTEA,
    witness_data    BYTEA[],
    sequence_no     BIGINT          NOT NULL DEFAULT 0xFFFFFF
FE,
    PRIMARY KEY (txid, vin_idx)
) PARTITION BY RANGE (block_height);

CREATE TABLE IF NOT EXISTS tx_inputs_0_100k    PARTITION OF t
x_inputs FOR VALUES FROM (0)       TO (100000);
CREATE TABLE IF NOT EXISTS tx_inputs_100k_200k  PARTITION OF
tx_inputs FOR VALUES FROM (100000)  TO (200000);
CREATE TABLE IF NOT EXISTS tx_inputs_200k_300k  PARTITION OF
tx_inputs FOR VALUES FROM (200000)  TO (300000);
CREATE TABLE IF NOT EXISTS tx_inputs_300k_400k  PARTITION OF
tx_inputs FOR VALUES FROM (300000)  TO (400000);
CREATE TABLE IF NOT EXISTS tx_inputs_400k_500k  PARTITION OF
tx_inputs FOR VALUES FROM (400000)  TO (500000);
CREATE TABLE IF NOT EXISTS tx_inputs_500k_600k  PARTITION OF
tx_inputs FOR VALUES FROM (500000)  TO (600000);
CREATE TABLE IF NOT EXISTS tx_inputs_600k_700k  PARTITION OF
tx_inputs FOR VALUES FROM (600000)  TO (700000);
CREATE TABLE IF NOT EXISTS tx_inputs_700k_800k  PARTITION OF
tx_inputs FOR VALUES FROM (700000)  TO (800000);
CREATE TABLE IF NOT EXISTS tx_inputs_800k_900k  PARTITION OF
tx_inputs FOR VALUES FROM (800000)  TO (900000);
```

```
CREATE TABLE IF NOT EXISTS tx_inputs_900k_1m    PARTITION OF
tx_inputs FOR VALUES FROM (900000)  TO (1000000);

CREATE INDEX IF NOT EXISTS idx_txin_txid   ON tx_inputs (txi
d);
CREATE INDEX IF NOT EXISTS idx_txin_prev   ON tx_inputs (prev
_txid, prev_vout) WHERE prev_txid IS NOT NULL;
CREATE INDEX IF NOT EXISTS idx_txin_height_brin ON tx_inputs
USING BRIN (block_height) WITH (pages_per_range=32);

-- ============================================================
======
-- 5. ADDRESS_TRANSACTIONS (denormalized, partitioned) ⭐
-- ============================================================
======
CREATE TABLE IF NOT EXISTS address_transactions (
    address         TEXT            NOT NULL,
    block_height    INT             NOT NULL,
    tx_index        INT             NOT NULL,
    txid            BYTEA           NOT NULL,
    role            SMALLINT        NOT NULL DEFAULT 0,   --
0=receiver,1=sender,2=both
    net_value_sats  BIGINT          NOT NULL DEFAULT 0,
    PRIMARY KEY (address, block_height, tx_index, txid)
) PARTITION BY RANGE (block_height);

CREATE TABLE IF NOT EXISTS address_tx_0_100k    PARTITION OF
address_transactions FOR VALUES FROM (0)       TO (100000);
CREATE TABLE IF NOT EXISTS address_tx_100k_200k  PARTITION OF
address_transactions FOR VALUES FROM (100000)  TO (200000);
CREATE TABLE IF NOT EXISTS address_tx_200k_300k  PARTITION OF
address_transactions FOR VALUES FROM (200000)  TO (300000);
CREATE TABLE IF NOT EXISTS address_tx_300k_400k  PARTITION OF
address_transactions FOR VALUES FROM (300000)  TO (400000);
CREATE TABLE IF NOT EXISTS address_tx_400k_500k  PARTITION OF
address_transactions FOR VALUES FROM (400000)  TO (500000);
```

```sql
CREATE TABLE IF NOT EXISTS address_tx_500k_600k  PARTITION OF
address_transactions FOR VALUES FROM (500000)  TO (600000);
CREATE TABLE IF NOT EXISTS address_tx_600k_700k  PARTITION OF
address_transactions FOR VALUES FROM (600000)  TO (700000);
CREATE TABLE IF NOT EXISTS address_tx_700k_800k  PARTITION OF
address_transactions FOR VALUES FROM (700000)  TO (800000);
CREATE TABLE IF NOT EXISTS address_tx_800k_900k  PARTITION OF
address_transactions FOR VALUES FROM (800000)  TO (900000);
CREATE TABLE IF NOT EXISTS address_tx_900k_1m    PARTITION OF
address_transactions FOR VALUES FROM (900000)  TO (1000000);

CREATE INDEX IF NOT EXISTS idx_addrtx_addr
    ON address_transactions (address, block_height DESC, tx_i
ndex DESC);
CREATE INDEX IF NOT EXISTS idx_addrtx_height_brin
    ON address_transactions USING BRIN (block_height) WITH (p
ages_per_range=32);

-- ============================================================
======
-- 6. UTXO_SET (hot table — current unspent outputs) ⭐
-- ============================================================
======
CREATE TABLE IF NOT EXISTS utxo_set (
    txid            BYTEA           NOT NULL,
    vout_idx        INT             NOT NULL,
    address         TEXT            NOT NULL,
    value_sats      BIGINT          NOT NULL,
    block_height    INT             NOT NULL,
    script_type     SMALLINT        NOT NULL DEFAULT 0,
    PRIMARY KEY (txid, vout_idx)
);

CREATE INDEX IF NOT EXISTS idx_utxo_address
    ON utxo_set (address, value_sats DESC);
CREATE INDEX IF NOT EXISTS idx_utxo_height_brin
```

```sql
    ON utxo_set USING BRIN (block_height) WITH (pages_per_ran
ge=32);

-- ============================================================
======
-- 7. ADDRESS_BALANCES (cached)
-- ============================================================
======
CREATE TABLE IF NOT EXISTS address_balances (
    address             TEXT        PRIMARY KEY,
    balance_sats        BIGINT      NOT NULL DEFAULT 0,
    total_received_sats BIGINT      NOT NULL DEFAULT 0,
    total_sent_sats     BIGINT      NOT NULL DEFAULT 0,
    utxo_count          INT         NOT NULL DEFAULT 0,
    tx_count            INT         NOT NULL DEFAULT 0,
    first_seen_height   INT,
    last_seen_height    INT,
    updated_at_height   INT         NOT NULL DEFAULT 0
);

-- ============================================================
======
-- 8. INDEX_STATE (indexer checkpoint / recovery)
-- ============================================================
======
CREATE TABLE IF NOT EXISTS index_state (
    id                  INT         PRIMARY KEY DEFAULT 1,
    last_indexed_height INT         NOT NULL DEFAULT 0,
    last_indexed_hash   BYTEA,
    started_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT single_row CHECK (id = 1)
);

INSERT INTO index_state (id, last_indexed_height) VALUES (1,
0)
```

```
ON CONFLICT DO NOTHING;

-- ============================================================
======
-- COMMENTS
-- ============================================================
======
COMMENT ON TABLE blocks IS 'One row per Bitcoin block';
COMMENT ON TABLE transactions IS 'All transactions, partition
ed by block_height';
COMMENT ON TABLE tx_outputs IS 'All transaction outputs (UTXO
s), spend status tracked here';
COMMENT ON TABLE tx_inputs IS 'All transaction inputs; join t
o tx_outputs via (prev_txid, prev_vout)';
COMMENT ON TABLE address_transactions IS 'Denormalized fast-p
ath table: one row per (address, tx) appearance. Use for addr
ess history queries.';
COMMENT ON TABLE utxo_set IS 'Current live UTXO set. Delete s
pent, insert new on each block. ~180M rows at current chain h
eight.';
COMMENT ON TABLE address_balances IS 'Cached address balance
s. Updated in batch per block group. Use for instant balance
lookup.';
```

Quick Reference: Which Table for Which Query

Query	Table	Index Used
Last N tx of address	`address_transactions`	PK (address, height DESC)
Current balance	`address_balances`	PK (address)
Current UTXOs of address	`utxo_set`	idx_utxo_address
Full tx detail	`transactions`	idx_tx_txid
Tx inputs + source addresses	`tx_inputs` JOIN `tx_outputs`	idx_txin_prev

Query	Table	Index Used
Tx outputs	`tx_outputs`	idx_txout_txid
Block info	`blocks`	PK (height) or idx_blocks_hash
All tx in height range	`transactions`	BRIN (block_height) + partition pruning
Wallet cluster (1-hop)	`tx_inputs` + `tx_outputs`	idx_txin_prev
Wallet cluster (3+ hops)	Apache AGE `bitcoin` graph	Cypher MATCH path